

Assignment #03 — Exercise 1

Smart Home Alarm System Apache Pekko Actor-Based Implementation

Alessandro Martini

Concurrent and Distributed Programming — ISI LM

University of Bologna, Cesena Campus

A.Y. 2025–2026

1 Problem Analysis

The Smart Home Alarm System is a reactive control system that monitors a network of peripheral sensors (motion detectors, door/window contacts) and manages an alarm lifecycle through a well-defined finite state machine. Users interact with the system via a numeric keypad for arming, disarming, and alarm-silencing operations.

1.1 Functional Overview

The system operates through five states:

1. **Disarmed** — sensor events are ignored; normal house activity.
2. **Exit Delay** — countdown after arming; sensors still inactive so the user can leave.
3. **Armed** — sensors are active; any detection in an active zone triggers entry delay.
4. **Entry Delay** — countdown after intrusion detection; entering the correct PIN disarms the system.
5. **Alarm** — siren is active; only a correct PIN can silence it.

The *bonus* requirement extends the system with **zone-based partial arming**: the house is divided into zones (LIVING_AREA, SLEEPING_AREA, PERIMETER, GROUND_FLOOR, UPPER_FLOOR), and only sensors in *active* zones can trigger the alarm.

1.2 State Machine

Figure 1 illustrates the alarm state machine with all transitions.

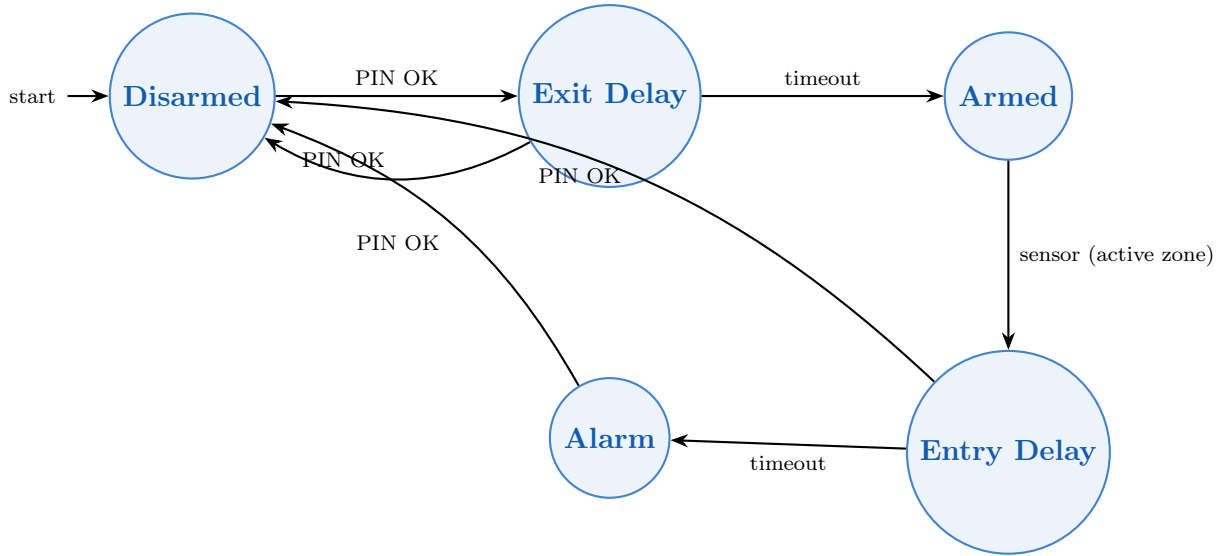


Figure 1: Finite state machine of the Smart Home Alarm System.

2 Concurrency Analysis

This problem is inherently concurrent: multiple sensors operate independently and simultaneously, a keypad accepts user input at any time, and timers fire asynchronously. Below we identify the key concurrent aspects and the challenges they pose.

2.1 Sources of Concurrency

Multiple Sensors

Each sensor (motion detector, door/window contact) is an independent entity that can fire events at any time. In a real deployment, dozens of sensors may trigger simultaneously, requiring the control unit to handle concurrent stimuli without data corruption or race conditions.

User Interaction

The keypad represents an additional concurrent input source. A user may enter a PIN while a sensor event is being processed, or during a timer countdown, creating interleaving scenarios that must be handled correctly.

Timer Management

Exit and entry delays are time-driven transitions. Timers run concurrently with message processing: a timer may expire while a PIN entry is being validated, or a sensor event may arrive during the last milliseconds of a countdown. These race conditions must be resolved deterministically.

State Consistency

The alarm state (`currentState`) and the set of active zones (`activeZones`) are mutable data that multiple concurrent events attempt to read or modify. Without proper isolation, concurrent access can lead to inconsistent states—e.g., processing a sensor event while the system is transitioning from EXIT DELAY to ARMED.

2.2 Concurrency Challenges

- C1. Race between timer expiration and user input.** If the entry delay timer expires at the same instant the user enters a correct PIN, the outcome must be deterministic: either the system disarms or the alarm triggers, but not both.
- C2. Simultaneous sensor events.** Multiple sensors may fire at the same time. The system must ensure that only the first event in ARMED state triggers the entry delay; subsequent events should have no additional effect.
- C3. Timer cancellation.** When the user disarms during a delay phase, the pending timer must be reliably cancelled. Failure to do so could cause a spurious transition after disarming.
- C4. Zone set mutation.** Switching between full and partial arming modifies the active zone set. This mutation must not interfere with concurrent sensor-event evaluation.

3 Adopted Strategy

The system is implemented using the **Actor Model**, specifically the **Apache Pekko** framework (typed API, Java). The actor model is a natural fit for this problem because it provides *message-driven concurrency with encapsulated state*, resolving all the challenges identified above.

3.1 Architecture Overview

The system is decomposed into three actor types:

Actor	Protocol	Responsibility
AlarmSystem	AlarmCommand	Central control unit: implements the FSM, manages timers, validates PINs, and emits events.
Sensor	Sensor.Command	Represents a physical sensor. On trigger, sends a SensorEvent to the alarm system.
EventLogger	AlarmEvent	Subscribes to alarm events for logging/auditing purposes.

Table 1: Actor decomposition.

3.2 Why the Actor Model Solves the Concurrency Challenges

Single-threaded message processing (mailbox)

Each actor processes *one message at a time* from its mailbox. This sequential processing within the **AlarmSystem** actor eliminates all race conditions on **currentState** and **activeZones**—no locks, no synchronization, no shared mutable state. Concurrent sensor events and user commands are serialized by the mailbox.

Timer integration via TimerScheduler

Pekko’s **TimerScheduler** delivers timer expirations as regular messages into the actor’s mailbox. This means timer callbacks are *interleaved with other messages*

in a deterministic, sequential order. If a `DisarmSystem` message arrives before `ExitDelayTimeout`, the disarm is processed first, the timer is cancelled, and the timeout message (if delivered) is harmlessly ignored because the state guard check fails.

Location transparency and isolation

Sensor actors communicate with the `AlarmSystem` exclusively through `ActorRef<AlarmCommand>` references. There is no shared memory between actors. Each sensor is a self-contained entity that can be spawned, stopped, or relocated independently.

Typed protocols

The use of Pekko's typed API with sealed interfaces (`AlarmCommand`, `AlarmEvent`) ensures at compile time that actors only receive messages they can handle, reducing the risk of protocol violations.

3.3 Key Design Decisions

3.3.1 State Machine as Conditional Logic

Rather than using Pekko's behavior-switching pattern (where each state is a separate `Behavior`), the implementation uses a single behavior with a `currentState` field and guard conditions on each message handler. This was chosen for simplicity: the state machine has only five states and the transition logic benefits from being in one cohesive place.

Listing 1: Guard-based state handling (excerpt).

```
1 private Behavior<AlarmCommand> onSensorEvent(SensorEvent cmd) {  
2     if (currentState == State.DISARMED  
3         || currentState == State.EXIT_DELAY) {  
4         return Behaviors.same(); // ignore  
5     }  
6     if (currentState == State.ARMED  
7         && activeZones.contains(cmd.zone())  
8         && cmd.triggered()) {  
9         startEntryDelay(cmd.zone());  
10    }  
11    return Behaviors.same();  
12 }
```

3.3.2 Timer Keys for Reliable Cancellation

Each timer is associated with a named key (`EXIT_TIMER_KEY`, `ENTRY_TIMER_KEY`). When the user disarms or re-arms, the corresponding timer is explicitly cancelled via `timers.cancel(key)`, preventing stale timeout messages from causing spurious transitions.

3.3.3 Zone-Based Partial Arming (Bonus)

Zones are modelled as a Java `EnumSet<Zone>`. Full arming activates `EnumSet.allOf(Zone.class)`, while partial arming uses a user-specified subset. Sensor events are filtered against `activeZones` before triggering transitions, enabling scenarios such as *night mode* (perimeter and ground floor active, upper floor inactive).

3.4 Message Flow

The typical interaction flow for an intrusion scenario is:

1. **User** → **AlarmSystem**: `ArmSystem("1234")` or `ArmSystemPartial("1234", zones)`
2. **AlarmSystem**: validates PIN, transitions to EXIT DELAY, starts exit timer
3. **Timer** → **AlarmSystem**: `ExitDelayTimeout` ⇒ transition to ARMED
4. **Sensor** → **AlarmSystem**: `SensorEvent(MOTION, LIVING_AREA, true)`
5. **AlarmSystem**: zone is active ⇒ transition to ENTRY DELAY, starts entry timer
6. **Timer** → **AlarmSystem**: `EntryDelayTimeout` ⇒ transition to ALARM
7. **User** → **AlarmSystem**: `StopAlarm("1234")` ⇒ transition to DISARMED

All steps occur through asynchronous message passing, with the mailbox guaranteeing sequential processing and state consistency.

3.5 Testing Strategy

The system is tested using **Pekko ActorTestKit** with **TestProbe** instances acting as event sinks. The test suite covers:

- Full arming lifecycle (Disarmed → Exit Delay → Armed)
- Intrusion sequence (Armed → Entry Delay → Alarm)
- Disarming during entry delay (race condition scenario)
- Disarming during exit delay (timer cancellation)
- Invalid PIN rejection at each state
- Sensor events ignored in Disarmed and Exit Delay states
- Zone-based filtering: inactive zones ignored, active zones trigger correctly
- Partial arming with mixed active/inactive zones

4 Conclusions

The actor model, as implemented via Apache Pekko, proved to be an excellent paradigm for the Smart Home Alarm System. The key insight is that the actor's mailbox provides a natural serialization point that eliminates the need for explicit synchronization, while timer messages are seamlessly integrated into the same sequential message flow. This design yields a system that is both *correct by construction* with respect to concurrency and *easy to reason about*, since the entire state machine logic resides within a single actor processing one message at a time.